

COMP3050 Team Project Assignment 2026 – API v1

Overview

The project involves building a server which tracks the state of a simple 2D tile-based virtual world. This virtual world could be used for playing a simple adventure game.

Students create and maintain the game world in a server, which communicates with a web-based client using a web API. This document specifies the API version number v1. (The API may change, and if it does the version number will be changed.)

Note: Students don't create the client or the tile images, which will all be supplied to the students. This is to avoid students getting distracted by the game play elements, because the intention is to focus on the server-side deployment and maintenance.

Game play

An example of game play might look like this:



Map

On the client window, the map view is shown with the player's character in the central location.

The player's view has a fixed size: an 11x11 square grid. But the world map may be larger.

The map has several elements:

- A character which can walk around within the map. A player will see the map with their own character positioned at the central tile. (Initially, only one character is supported, but multiplayer support may be added later.)
- Surfaces, which the player can walk on, such as grass, dirt, and various kinds of flooring.
- Impassable elements, such as walls, doors, and water.

Movement

Characters can be moved one space in one of the four cardinal directions:

Direction	Direction name	dy	dx	Description	Client key
N	North	-1	0	One space north.	W or ↑
W	West	0	-1	One space west.	A or ←
S	South	+1	0	One space south.	S or ↓
E	East	0	+1	One space east.	D or →

Diagonal motion and moving more than one space are disallowed.

API

The API supports several operations:

/move	?dy=DY &dx=DX	Move the character in the corresponding direction if possible. Locations are relative to the character, so dy=0&dx=+1 is E one space, and dy=+1&dx=0 is S one space. Returns 200 with confirmation of the new location if successful, or 204 if motion is blocked or invalid (e.g. if moving diagonally or more than one space).
/info	?y=Y &x=X	Fetch information about a particular space, including floor tile kind. The location given in the y and x parameters are the character's absolute location in the map. The y co-ordinate increases in the South direction, the x co-ordinate increases in the East direction. Return 200 with JSON information if successful, or 204 if obtaining information about that tile is not allowed (e.g. if the character is not at the location specified).

MOVE requests

The central location where the player's character appears in the view is assigned the number (0,0) in the relative (DY,DX) coordinate system used by MOVE operations. Movement requests for game spaces are relative to character's current location, i.e. relative to that (0,0) view location.

One dy and one dx parameter may be given in each MOVE request. A missing parameter is assumed to be zero. So the following are valid:

Request	Validity	Notes
/move?dy=-1&dx=0	valid	Move N.
/move?dy=-1	valid	Defaults dx=0
/move?dx=+1	valid	Defaults dy=0
/move?dy=0&dx=0	valid	Valid but does nothing. No movement.
/move	valid	Defaults dy=0 and dx=0. No movement.

But the following are invalid:

Request	Validity	Notes
/move?dy=+1&dx=+1	invalid	Diagonal motion is invalid.
/move?dy=-2&dx=0	invalid	Moving more than one space is invalid.

Move requests may fail for various reasons, such as going beyond the edge of the map, or intersecting a blocking map element such as a brick wall. In such cases, a 204 status code is returned with an empty response body, and no change is made to the server's recorded location of the player's character.

Valid move requests which succeed cause the server's recorded location of the player's character to be updated to the new location, and a 200 status code to be returned, with a JSON response body reporting the new absolute map x and y location of the player's character. For example:

```
{
  "y": 0,
  "x": 6
}
```

INFO requests

The client uses the INFO request to discover information about locations nearby the character. An INFO request uses absolute (Y,X) locations, relative to the map's co-ordinate system. The y co-ordinate increases in the South direction, the x co-ordinate increases in the East direction.

If an invalid INFO request is made, the server returns a 204 status code, and the message body will be empty. A request is invalid if the requested x and y values don't match the current player character location.

Upon success, an INFO request returns a 200 status, and the message body will contain JSON data which describes the specified space.

The INFO request's JSON response specifies:

- The "info", a rectangular array (an array of arrays) of floor tile kinds, which the client uses to display the appropriate image tile at that location. Each floor tile kind is a short string.
- The "y" and "x" location that were requested, as integers (which correspond with the character's location).
- The "top", "left", "bottom", and "right" (inclusive) edges locations of the returned rectangular array, as integers.

An example INFO request may be:

```
/info?y=7&x=5
```

An example of an INFO request's JSON response with status 200 might be:

```
{
  "y": 7,
  "x": 5,
  "top": 2,
  "left": 0,
  "bottom": 12,
  "right": 10,
  "info": [
    ["S", "w", "w", "w", "w", "S", "g", "g", "g", "W", "W"],
    ["S", "w", "w", "w", "w", "S", "g", "g", "W", "W", "g"],
    ["S", "S", "S", "w", "S", "S", "g", "g", "W", "g", "g"],
    ["g", "g", " ", " ", " ", "g", "g", "g", "W", "W", "W", "g"],
    [".", "g", "g", " ", " ", "g", "g", "g", "W", "g", "W", "."],
    [ " ", " ", " ", " ", " ", "g", "g", "W", "W", "g", "W", "W"],
    [ "_", " ", " ", " ", " ", "g", "g", "W", "W", "g", "g", "W"],
    [ "_", "g", "t", "t", "g", "W", "W", "W", "g", "g", "g", "W"],
    [ "_", "g", "g", "t", "g", "W", "W", "g", "g", "g", "g", "W"],
    [ "_", "g", "g", "g", "g", "g", "g", "g", "g", "g", "g", "W"],
    [ "_", "g", "g", "g", "g", "g", "g", "t", "t", "g", "W", "W"],
    [ "_", "g", "g", "g", "g", "g", "g", "t", "g", "g", "W", " "]
  ]
}
```

Some floor kinds block movement. For example, a brick wall prevents a character from moving into that location. Similarly, water prevents character motion.

Floor tile kinds are indicated using short strings. Each letter or symbol in a string specify a tile kind:

Letter or symbol	Tile kind	Blocks movement?
B	brick wall	blocking
b	bridge	-
D	doors closed	blocking
d	doors opened	-
_	dirt	-
f	flagstones	-
g	grass	-
p	pebbles	-
.	one rock	-
,	two rocks	-
:	three rocks	-
;	six rocks	-
S	stone wall	blocking
s	sand	-
t	tree	-
W	water waves	blocking
w	wooden boards	-

The server ensures that MOVE requests fail if that would cause the player's character to intersect a **blocking** map element.

Note that uppercase letters have been used for all **blocking** map elements, but that is a convenience, not a strict rule, and it could change in future versions of this document.

Client-side View

The client displays an 11x11 grid of square locations.

Each location is drawn within the client in the following way:

1. First, the INFO command is used to discover information about that location.
2. Then each floor tile is drawn.
3. Any character at a location is then drawn over the tile.

Vision of a tile is restricted if the location is too far away (more than 5 spaces away in Y or X).

Because a tile is drawn then a character may be drawn over the top, transparent pixels in the character's sprite will reveal the pixels of the tile below.

For speed, when the player moves the character, a check for whether the tile is blocking is performed on the client-side. If the motion would succeed on the client's copy of the map, a MOVE request is made only after this check. This avoids a potentially costly round-trip to the server for every move. However, the client should undo any moves that fail as a result of 204 status codes.

Server-side Map

The server software must maintain a map, and serve rectangular pieces of it to the client as requested (in response to each INFO request).

A simple way to store a map is to use a text file containing the letters/symbols used for floor tile kinds. This text file can be loaded into memory when the server starts, e.g. as a rectangular array of bytes. This avoids the need for a database or anything too complex.

Versions

This document is at version 1. This is the initial specification. It may be incomplete or lack features.

Later versions of this document will supersede earlier versions.